04. Divide-and-Conquer

Hyunjong Choi, Ph.D.
Assistant Professor of Computer Science
San Diego State University

Outline

- More algorithms based on ***divide-and-conquer***
  - Maximum-subarray problem (Textbook 3rd edition 68 – 74)
  - Matrix multiplication problem (straightforward method vs. Strassen's algorithm) (Textbook 3rd edition 75 – 82)

- Solving recurrence equations
  - ***Substitution method*** for solving recurrence (Textbook 3rd edition 83 – 87)
  - ***Recursion-tree method*** (Textbook 3rd edition 88 – 92)
  - Master method (master theorem) (Textbook 3rd edition 93 – 106)

Master method (theorem)

❑ It provides a "*cookbook*" for solving recurrences of the form

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

- where $\boldsymbol{a \geq 1}$ ***and*** $\boldsymbol{b > 1}$ are constants and $f(n)$ is an asymptotically positive function.
- It divides a problem of size n into a subproblems, each of size n/b.
- $f(n)$: the cost of dividing and combining, called "***driving function***"

Master method (theorem)

Let a ≥ 1 and b > 1 are constants, and let $f(n)$ be a driving function that is asymptotically positive function. Define $T(n)$ by

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Then $T(n)$ has the following asymptotic bounds:

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af\left(\frac{n}{b}\right) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$.

Intuitive understanding of master method

- In each of the three cases, we compare $f(n)$ with the function $n^{\log_b a}$.

  Driving function **Watershed function**

  - **Case 1**, the watershed $n^{\log_b a}$ grows asymptotically faster than the driving function $f(n)$. Then $T(n) = \Theta(n^{\log_b a})$.

  - **Case 2**, the watershed $n^{\log_b a}$ and driving functions $f(n)$ grow at nearly the same asymptotic rate. Then, multiply by a logarithmic factor $T(n) = \Theta(f(n) \lg n) = \Theta\left(n^{\log_b a} \lg n\right)$

  - **Case 3**, the driving function $f(n)$ grow asymptotically faster than the watershed function $n^{\log_b a}$. Then $T(n) = \Theta(f(n))$.

Some technical details

- In **case 1**, $f(n)$ must be smaller than $n^{\log_b a}$ and it must be ***polynomially*** smaller.
  - $n^{\log_b a}$ must be asymptotically larger than the $f(n)$ at least by a factor of $\Theta(n^{\epsilon})$ for some constant $\epsilon > 0$.
- In **case 3**, $f(n)$ must be greater than $n^{\log_b a}$ and it must be ***polynomially*** larger.
  - $f(n)$ must be asymptotically larger than $n^{\log_b a}$ at least by a factor of $\Theta(n^{\epsilon})$ for some constant $\epsilon > 0$.
  - In addition, "regularity" condition that $af\left(\frac{n}{b}\right) \leq cf(n)$
- The three cases do not cover all the possibilities for $f(n)$.
  - There is a gap between case 1 and 2 when $f(n)$ is smaller than $n^{\log_b a}$, but not polynomially smaller.
  - There is a gap between case 2 and 3 when $f(n)$ is larger than $n^{\log_b a}$, but not polynomially larger.

Example 1

❑ Maximum subarray problem (and merge sort)

$$T(n) = \begin{cases} \Theta(1), & \text{if } n = 1 \\ 2T(n/2) + \Theta(n), & \text{if } n > 1 \end{cases}$$

- We have $a = 2, b = 2, f(n) = \Theta(n)$.
- Compare driving & watershed function
  - ✓ We have $n^{\log_b a} = n = \Theta(n)$

- ⇨ **case 2** applies.
- Therefore, the solution is $T(n) = \Theta(n \lg n)$.

Let $a > 0$ and b > 1 are constants, and let $f(n)$ be a driving function that is asymptotically positive function. Define $T(n)$ by

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Then $T(n)$ has the following asymptotic bounds:

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af\left(\frac{n}{b}\right) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$.

Example 2

❑ Strassen's algorithm

$$T(n) = \begin{cases} \Theta(1), & \text{if } n = 1 \\ 7T(n/2) + \Theta(n^2), & \text{if } n > 1 \end{cases}$$

- We have $a = 7$, $b = 2$, $f(n) = \Theta(n^2)$.
- Watershed function: we have $n^{\log_b a} = n^{2.81} = \Theta(n^{2.81})$
- Then, driving function is *polynomially* smaller by a factor of $n^{0.81}$.

- ⇨ **case 1** apples with $\epsilon = 0.81$
- Therefore, the solution is $T(n) = \Theta(n^{2.81})$.

Let $a > 0$ and b > 1 are constants, and let $f(n)$ be a driving function that is asymptotically positive function. Define $T(n)$ by

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Then $T(n)$ has the following asymptotic bounds:

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af\left(\frac{n}{b}\right) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$.

Example 3

Let $a > 0$ and b > 1 are constants, and let $f(n)$ be a driving function that is asymptotically positive function. Define $T(n)$ by

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Then $T(n)$ has the following asymptotic bounds:

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af\left(\frac{n}{b}\right) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$.

- Consider the recurrence $T(n) = 2T\left(\frac{n}{2}\right) + n \lg n$
  - $a = 2, b = 2$, watershed function: $n^{\log_b a} = n$
  - Which means $f(n) = n \lg n$ is asymptotically larger than $n^{\log_b a}$.

Can we apply case 3 ?

  - It looks like so, but the problem is that $f(n)$ is **not** *polynomially* larger.

  - Can we find a $\epsilon > 0$, such that $f(n) \geq n^{\log_b a + \epsilon}$
  - i.e., $n \lg n \geq n^{1+\epsilon} \Rightarrow \lg n \geq cn^{\epsilon}$
  - However, we cannot find such a c, because $\lg n$ grows slower than any polynomial function n^{ϵ} when $\epsilon > 0$.
  - Therefore, we cannot apply case 3 (i.e., master theorem).

Example 4

- Consider $T(n) = 9T\left(\frac{n}{3}\right) + n$
  - $a = 9, b = 3$, watershed function: $n^{\log_b a} = n^2$
  - $f(n) = n = O\left(n^{\log_b a - \epsilon}\right) = O(n^{2-\epsilon})$, when $\epsilon = 1$
  - **Case 1** can be applied
  - Therefore, $T(n) = \Theta(n^2)$.

Let $a > 0$ and b > 1 are constants, and let $f(n)$ be a driving function that is asymptotically positive function. Define $T(n)$ by

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Then $T(n)$ has the following asymptotic bounds:

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af\left(\frac{n}{b}\right) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$.

Example 5

- Consider $T(n) = T\left(\frac{2n}{3}\right) + 1$
  - $a = 1, b = 3/2$, watershed function: $n^{\log_b a} = n^{\log_{3/2} 1} = n^0 = 1 = f(n)$
  - **Case 2** applies.
  - Therefore, the solution is $T(n) = \Theta(\lg n)$.

Let $a > 0$ and b > 1 are constants, and let $f(n)$ be a driving function that is asymptotically positive function. Define $T(n)$ by

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Then $T(n)$ has the following asymptotic bounds:

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af\left(\frac{n}{b}\right) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$.

Example 6

- Consider $T(n) = 3T\left(\frac{n}{4}\right) + n \lg n$
  - $a = 3, b = 4$, watershed function: $n^{\log_b a} = n^{\log_4 3} = O(n^{0.793})$
  - Since $f(n) = n \lg n = \Omega(n^{\log_4 3 + \epsilon})$ is satisfied as long as $\epsilon < 0.207$, because $\lg n$ is smaller than any polynomial function

  - Then **case 3** applies if we can also show that the regularity condition holds
  - $af\left(\frac{n}{b}\right) = 3\left(\frac{n}{4}\right)\lg(\frac{n}{4}) = \frac{3}{4}n \lg n - \frac{3n}{2} \leq cf(n) = cn \lg n$
  - The above holds if $c = \frac{3}{4}$, for sufficiently large n.
  - Therefore, the solution is $T(n) = \Theta(n \lg n)$.

Let $a > 0$ and b > 1 are constants, and let $f(n)$ be a driving function that is asymptotically positive function. Define $T(n)$ by

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Then $T(n)$ has the following asymptotic bounds:

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af\left(\frac{n}{b}\right) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$.

Optional

Proof of the master theorem

- The proof has two steps: lemma 1 and lemma 2.
  - Lemma 1: reduces the problem of solving the recurrence to the problem of evaluating an expression that contains a *summation*.
  - Lemma 2 determines *bounds* on this summation.

Lemma 1: Let $a \geq 1$ and $b > 1$ be constants, and let $f(n)$ be a nonnegative function defined on exact power of b (simplification). Define $T(n)$ as:

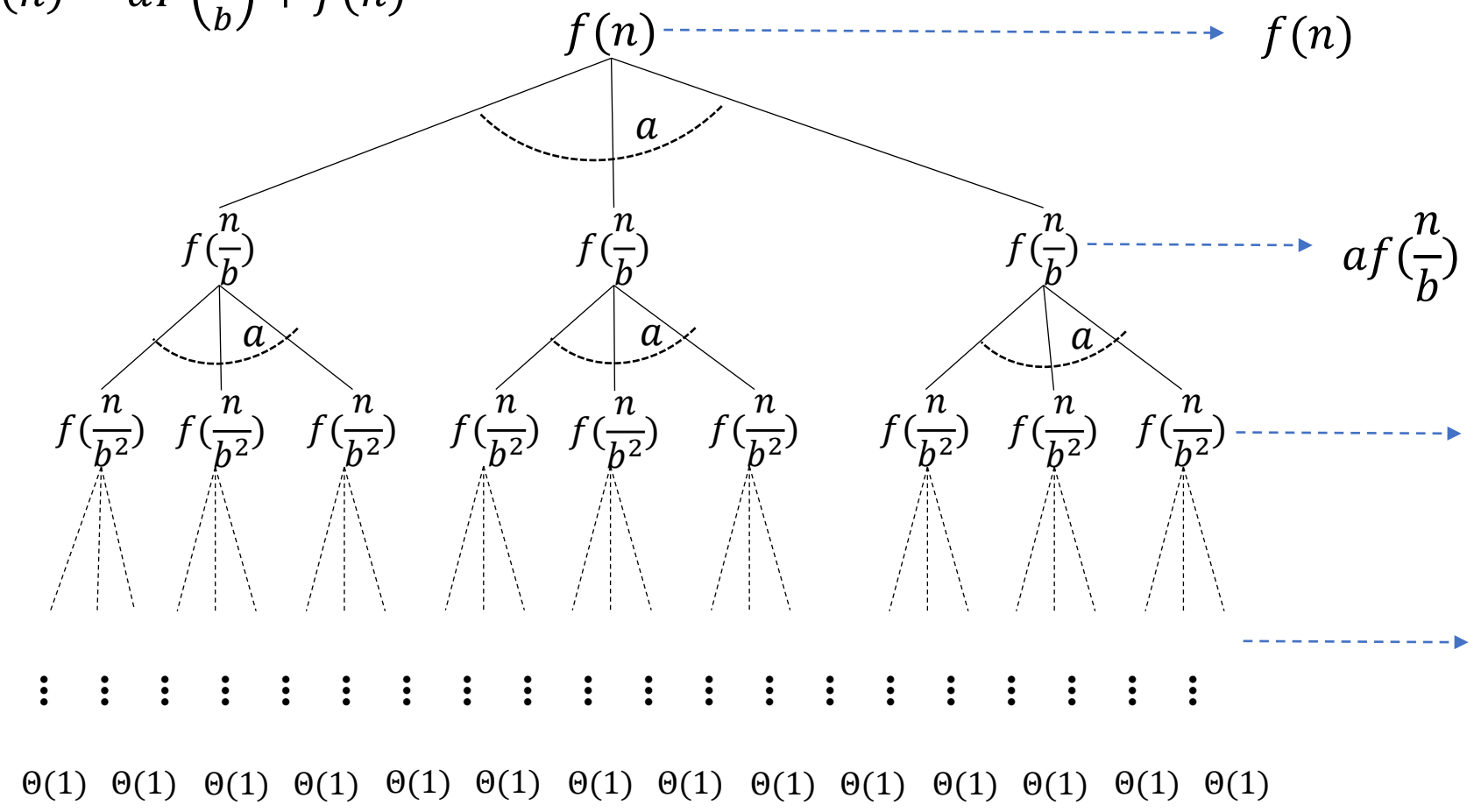
$$T(n) = \begin{cases} \Theta(1), & \text{if } n = 1 \\ aT\left(\frac{n}{b}\right) + f(n), & \text{if } n = b^i \end{cases}$$

$$T(n) = \Theta\left(n^{\log_b a}\right) + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$$

Optional

Lemma 1. Proof

- $T(n) = aT\left(\frac{n}{b}\right) + f(n)$



- At depth j
- there are a^j nodes
- each has cost $f(\frac{n}{b^j})$

$$a^j f(\frac{n}{b^j})$$

Optional

Lemma 1. Proof

- Leaf nodes
  - When $\frac{n}{b^j} = 1$, ⇨ $j = \log_b n$, ⇨ Leaf nodes are at depth $\log_b n$
  - There are $a^{\log_b n} = n^{\log_b a}$ leaf nodes
  - Sum of all leaf nodes: $n^{\log_b a} \cdot \Theta(1) = \Theta(n^{\log_b a})$

- Internal nodes
  - Depth j ranges from 0 to $\log_b n - 1$
  - Sum of all internal nodes:

$$\sum_{j=0}^{\log_b n - 1} a^j f(\frac{n}{b^j})$$

$$T(n) = \Theta\left(n^{\log_b a}\right) + \sum_{j=0}^{\log_b n - 1} a^j f(\frac{n}{b^j})$$

⇨ Lemma 1 is proved

Optional

What does Lemma 1 tell us?

Equation 1.

$$T(n) = \underbrace{\Theta\left(n^{\log_b a}\right)}_{\text{The costs of solving } n^{\log_b a} \text{ subproblems of size 1}} + \underbrace{\sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)}_{\text{The costs of dividing and combining}}$$

- We want to know the asymptotic bounds on this part

- The costs of solving $n^{\log_b a}$ subproblems of size 1

- The costs of dividing and combining

❑ Master theorem describes how the total cost is distributed:

- ✓ Case 1: dominated by the costs in leaf nodes
- ✓ Case 2: evenly distributed among all levels of the recursion tree
- ✓ Case 3: dominated by the cost of the root

Optional

Lemma 2

Lemma 2: Let $g(n) = \sum_{j=0}^{\log_b n-1} a^j f(\frac{n}{b^j})$, it has the following asymptotic bounds:

1. If $f(n) = O(n^{\log_b a-\epsilon})$ for some constant $\epsilon > 0$, then $g(n) = O(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $g(n) = \Theta(n^{\log_b a} \lg n)$.
3. If $af\left(\frac{n}{b}\right) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n, then $g(n) = \Theta(f(n))$.

Optional

Proof of Lemma 2 (case 1)

- For case 1, $f(n) = O(n^{\log_b a-\epsilon})$ implies that $f\left(\frac{n}{b^j}\right) = O(\left(\frac{n}{b^j}\right)^{\log_b a-\epsilon})$.

  Substituting into $g(n)$:

$$g(n) = \sum_{j=0}^{\log_b n-1} a^j f(\frac{n}{b^j}) = O\left(\sum_{j=0}^{\log_b n-1} a^j \left(\frac{n}{b^j}\right)^{\log_b a-\epsilon}\right)$$

$$\Rightarrow n^{\log_b a-\epsilon} \sum_{j=0}^{\log_b n-1} \left(a \cdot \frac{b^\epsilon}{b^{\log_b a}}\right)^j = n^{\log_b a-\epsilon} \sum_{j=0}^{\log_b n-1} (b^\epsilon)^j$$

$$= n^{\log_b a-\epsilon} \left(\frac{b^{\epsilon \log_b n} - 1}{b^\epsilon - 1}\right)$$

$$= n^{\log_b a-\epsilon} \left(\frac{n^\epsilon - 1}{b^\epsilon - 1}\right) = n^{\log_b a-\epsilon} O(n^\epsilon) = O(n^{\log_b a})$$

Optional

Proof of Lemma 2 (case 2)

- For case 2, $f(n) = \Theta(n^{\log_b a})$ implies that $f\left(\frac{n}{b^j}\right) = \Theta(\left(\frac{n}{b^j}\right)^{\log_b a})$. Substituting into $g(n)$:

$$g(n) = \sum_{j=0}^{\log_b n - 1} a^j f(\frac{n}{b^j}) = \Theta\left(\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a}\right)$$

$$\Rightarrow n^{\log_b a} \sum_{j=0}^{\log_b n - 1} \left(\frac{a}{b^{\log_b a}}\right)^j$$

$$= n^{\log_b a} \sum_{j=0}^{\log_b n - 1} (1)^j$$

$$= n^{\log_b a} \cdot \log_b n$$

$$= \Theta(n^{\log_b a} \lg n)$$

Optional

Proof of Lemma 2 (case 3)

- For case 3, $af\left(\frac{n}{b}\right) \leq cf(n) \rightarrow f\left(\frac{n}{b}\right) \leq \frac{c}{a}f(n)$
- Iterate many times: $f\left(\frac{n}{b^2}\right) \leq \frac{c}{a}f\left(\frac{n}{b}\right)$, $f\left(\frac{n}{b^3}\right) \leq \frac{c}{a}f\left(\frac{n}{b^2}\right)$, …, $f\left(\frac{n}{b^i}\right) \leq \frac{c}{a}f\left(\frac{n}{b^{i-1}}\right)$

$\rightarrow f\left(\frac{n}{b^i}\right) \leq \left(\frac{c}{a}\right)^i f(n)$, or $a^i f\left(\frac{n}{b^i}\right) \leq c^i f(n)$. Substituting into $g(n)$:

$$g(n) = \sum_{j=0}^{\log_b n-1} a^j f(\frac{n}{b^j}) \leq \sum_{j=0}^{\log_b n-1} c^i f(n) \leq f(n)\sum_{j=0}^{\infty} c^i \quad = f(n)\frac{1}{1-c} = O(f(n))$$

- From the form of $g(n)$, we know that $g(n) = \Omega(f(n))$
- Therefore, $g(n) = \Theta(f(n))$

➡ Lemma 2 is proved!

Optional

Combining Lemma 1 and 2

- Lemma 1 tells: $T(n) = \Theta\left(n^{\log_b a}\right) + g(n)$
  - **Case 1**, $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$,
  - $T(n) = \Theta\left(n^{\log_b a}\right) + O\left(n^{\log_b a}\right) = \Theta(n^{\log_b a})$

  - **Case 2**, $f(n) = \Theta(n^{\log_b a})$,
  - $T(n) = \Theta\left(n^{\log_b a}\right) + \Theta\left(n^{\log_b a} \lg n\right) = \Theta\left(n^{\log_b a} \lg n\right)$

  - **Case 3**, $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$,
  - $T(n) = \Theta\left(n^{\log_b a}\right) + \Theta(f(n)) = \Theta(f(n))$

➡ Master theorem proved!

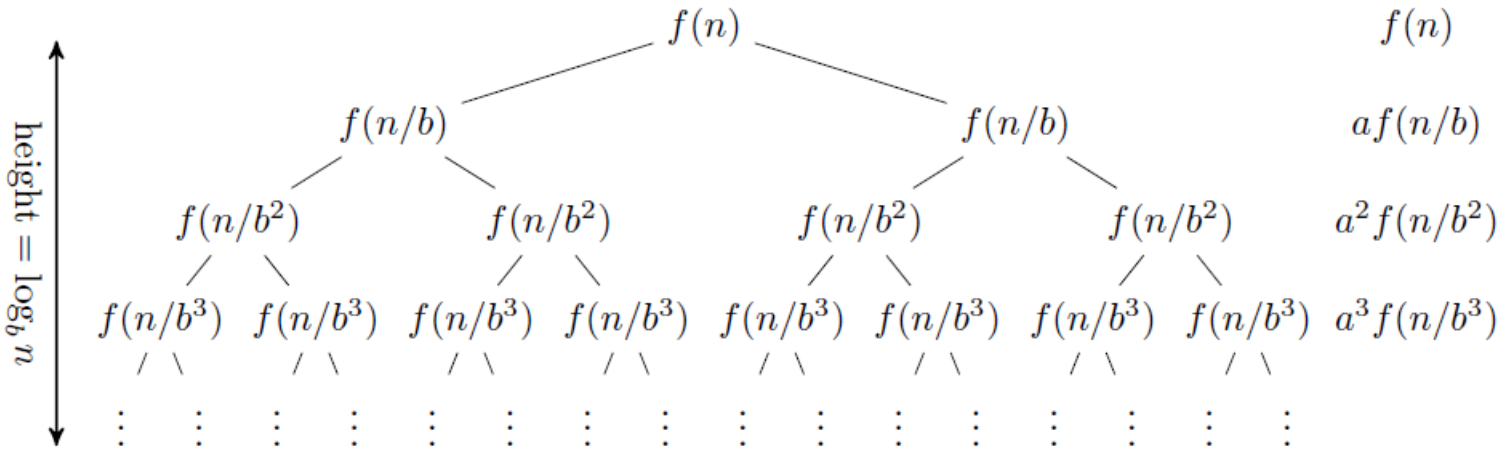
Optional

Proof of master method by math

< Master method >

Consider the recurrence $T(n) = aT\left(\frac{n}{b}\right) + f(n)$, where $a \geq 1, b > 1$ are constants. Then,

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af\left(\frac{n}{b}\right) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$.



$$T(n) = \sum_{i=0}^{\log_b n} a^i f\left(\frac{n}{b^i}\right) + O(n^{\log_b a})$$

Equation 1.

Optional

Proof of master method

❑ Proof of case 1,

$$T(n) = \sum_{i=0}^{\log_b n} a^i f\left(\frac{n}{b^i}\right) + O(n^{\log_b a}) \leq \sum_{i=0}^{\log_b n} a^i f\left(\frac{n}{b^i}\right)^{\log_b a - \varepsilon} + O(n^{\log_b a})$$

And

$$\sum_{i=0}^{\log_b n} a^i f\left(\frac{n}{b^i}\right)^{\log_b a - \varepsilon} = n^{\log_b a - \epsilon} \sum_{i=0}^{\log_b n} a^i b^{-i log_b a} b^{i\epsilon}$$

$$= n^{\log_b a - \epsilon} \sum_{i=0}^{\log_b n} a^i a^{-i} b^{i\epsilon} = n^{\log_b a - \epsilon} \sum_{i=0}^{\log_b n} b^{i\epsilon}$$

$$= n^{\log_b a - \epsilon} \times \frac{b^{\epsilon(\log_b n + 1)} - 1}{b^\epsilon - 1} = n^{\log_b a - \epsilon} \times \frac{n^\epsilon b^\epsilon - 1}{b^\epsilon - 1}$$

$$\leq n^{\log_b a - \epsilon} \times \frac{n^\epsilon b^\epsilon}{b^\epsilon - 1} = n^{\log_b a} \times \frac{b^\epsilon}{b^\epsilon - 1}$$

$$= O(n^{\log_b a})$$

Therefore, $\boldsymbol{T(n) = O(n^{\log_b a})}$

Optional

Proof of master method (cont.)

❑ Proof of case 2,

We have

$$\sum_{i=0}^{\log_b n} a^i \left(\frac{n}{b^i}\right)^{\log_b a} = n^{\log_b a} \sum_{i=0}^{\log_b n} a^i b^{-i log_b a} = n^{\log_b a} \sum_{i=0}^{\log_b n} a^i a^{-i}$$
$$= n^{\log_b a}(\log_b n + 1) = \Theta(n^{\log_b a} \log n)$$

Now,

$$T(n) = \sum_{i=0}^{\log_b n} a^i f\left(\frac{n}{b^i}\right) + O(n^{\log_b a}) = \sum_{i=0}^{\log_b n} a^i \left(\frac{n}{b^i}\right)^{\log_b a} + O(n^{\log_b a})$$
$$= \Theta\left(n^{\log_b a} \log n\right) + O\left(n^{\log_b a}\right) = \boldsymbol{\Theta\left(n^{log_b a} log\, n\right)}$$

Optional

Proof of master method (cont.)

- Case 3: If $f(n) = \Omega(n^{\log_b a+\epsilon})$ for some constant $\epsilon > 0$, and if $af\left(\frac{n}{b}\right) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$.
- Proof of case (C),
  - The lower bound is immediate, because $f(n)$ is a term of the Equation 1.
  - The upper bound, we use the smoothness condition, satisfied by

  $f(n) = n^{\log_b a+\epsilon}$ for any $\epsilon > 0$ with $c = b^{-\epsilon} < 1$:

$$af\left(\frac{n}{b}\right) = a\left(\frac{n}{b}\right)^{\log_b a+\epsilon} = an^{\log_b a+\epsilon}b^{-\log_b a}b^{-\epsilon} = f(n)b^{-\epsilon}.$$

  In this case, we have $a^i f\left(\frac{n}{b^i}\right) \leq c^i f(n)$ (easy induction on i using the smoothness condition).

Therefore,

$$T(n) = \sum_{i=0}^{\log_b n} a^i f\left(\frac{n}{b^i}\right) + O\left(n^{\log_b a}\right) \leq \sum_{i=0}^{\log_b n} c^i f(n) + O\left(n^{\log_b a}\right)$$

$$\leq f(n)\sum_{i=0}^{\infty} c^i + O\left(n^{\log_b a}\right) = f(n) \times \frac{1}{1-c} + O\left(n^{\log_b a}\right) = O(f(n))$$

Example

Problem

- Solve the following recurrence by master theorem. If it cannot be solved by master theorem, just mark N/A.

1) $T(n) = \sqrt{2} \times T\left(\frac{n}{2}\right) + \log n$

2) $T(n) = 0.5 \times T\left(\frac{n}{2}\right) + \frac{1}{n}$